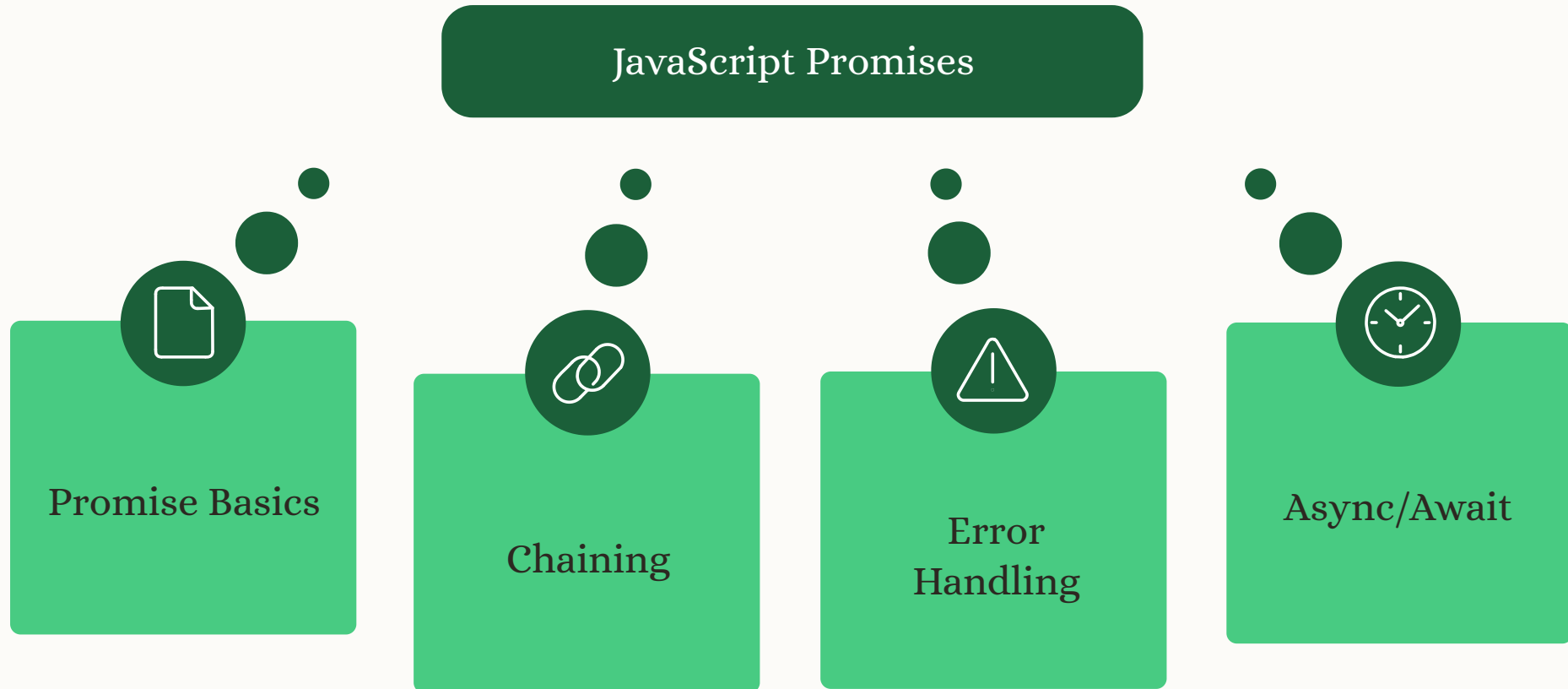


The Promise Object and Its States

A fundamental concept in asynchronous JavaScript programming for managing asynchronous operations with cleaner, more maintainable code.



Introduction to the Promise Object

A Promise is an object representing the eventual completion or failure of an asynchronous operation. It serves as a placeholder for a value that may not be available immediately but will be resolved at some point in the future.

Purpose

Promises provide a structured approach to handling asynchronous operations, eliminating the complexity of callback functions and improving code readability.

Key Advantage

They allow you to attach handlers to asynchronous operations, enabling sequential chaining of operations and better error management compared to traditional callback patterns.

Common Use Cases

Network requests, file operations, database queries, and any operation where the result isn't immediately available require asynchronous handling.

Promises solve the "callback hell" problem by providing a cleaner syntax for handling multiple asynchronous operations in sequence, making code more maintainable and easier to debug.



Promise Life Cycle and Execution

Promise Creation

A Promise is created using the Promise constructor, which takes an executor function with two parameters: resolve and reject. This function executes immediately when the Promise is instantiated.

```
const myPromise = new Promise(  
  (resolve, reject) => {  
    // Asynchronous operation  
    if (success) {  
      resolve(value);  
    } else {  
      reject(error);  
    }  
  }  
);
```

Execution Flow

When created, the Promise begins in a pending state. The executor function runs synchronously, whilst the asynchronous operation proceeds. Once the operation completes, the Promise transitions to either fulfilled or rejected state, triggering the appropriate handlers.

The transition between states is permanent and irreversible—a Promise can only settle once, ensuring predictable behaviour in asynchronous workflows.



The Three States of a Promise

01

Pending

The initial state when a Promise is created. The asynchronous operation has neither completed successfully nor failed. The Promise remains in this state until the operation finishes.

- Operation is still in progress
- No result or error yet available
- Can transition to fulfilled or rejected

02

Fulfilled

The state indicating successful completion of the asynchronous operation. The Promise now holds the resulting value, which can be accessed through handler functions.

- Operation completed successfully
- Result value is available
- State is permanent and immutable

03

Rejected

The state indicating failure of the asynchronous operation. The Promise contains an error reason, typically an Error object, explaining why the operation failed.

- Operation encountered an error
- Error reason is available
- State is permanent and immutable

❏ **Important:** Once a Promise settles (becomes fulfilled or rejected), it cannot change state again. This immutability ensures reliable asynchronous behaviour.



Promise Handling Methods and Summary



`.then()`

Handles the fulfilled state. Accepts a callback function that receives the resolved value as an argument. Returns a new Promise, enabling method chaining.

```
promise.then(value => {
  console.log(value);
});
```



`.catch()`

Handles the rejected state. Accepts a callback function that receives the error reason. Essential for error handling in Promise chains.

```
promise.catch(error => {
  console.error(error);
});
```



`.finally()`

Executes regardless of whether the Promise is fulfilled or rejected. Useful for cleanup operations that must occur in both scenarios.

```
promise.finally(() => {
  // Cleanup code
});
```

Key Learning Outcomes

- Understand that Promises represent asynchronous operations with three distinct states: pending, fulfilled, and rejected
- Recognise that Promises eliminate callback hell through chainable methods and cleaner syntax
- Apply `.then()`, `.catch()`, and `.finally()` methods appropriately to handle Promise results and errors
- Implement Promises to manage asynchronous operations with improved code maintainability and error handling

Promises are foundational to modern JavaScript development, enabling developers to write more elegant asynchronous code whilst maintaining better control over execution flow and error management.

